

GAME ENGINE BLACK BOOK

WOLFENSTEIN 3D

FABIEN SANGLARD

v2.2

Direitos autorais

Para ilustrar o funcionamento do motor gráfico de Wolfenstein 3D, algumas capturas de tela, imagens, sprites e texturas pertencentes à id Software e protegidas por direitos autorais são reproduzidas neste livro. Os seguintes itens são utilizados sob a doutrina do "uso justo":

1. Todas as capturas de tela do jogo, tela de título, tela de login, tela de carnificina total.
2. Todas as capturas de tela dos menus do jogo (menu principal, menu de som).
3. Todas as texturas de sequência 3D (parede azul, madeira, madeira escura, parede quadriculada).
4. Todos os sprites de sequência 3D (guarda marrom, guarda morto, cachorro morto).
5. Todas as imagens sequenciais 3D usadas para o HUD.
6. Todas as capturas de tela de Spear of Destiny, Catacomb 3-D e Hovortank One.

Agradecimentos

Agradecemos a John Carmack, John Romero, Romain Guy, Victoria Ho e Aurelien Sanglard pela generosa ajuda. Este projeto jamais teria se concretizado sem eles.

Agradecemos a Jim Leonard e Foone Turing, que disponibilizaram voluntariamente sua frota de computadores 286, 386 e placas VGA para realizar testes de desempenho precisos em Wolfenstein 3D.

Agradecemos a Jim Leonard por compartilhar seu conhecimento enciclopédico sobre arquitetura de sistemas de PC e programação. Sua paciência ao explicar sistemas de som e sistemas de memória estendida contribuiu para que este livro fosse publicado com informações precisas.

Agradecemos a Chet Haase, Daniel Thornburgh, Xiao Yu e Chris Forbes pela revisão e pela identificação de erros.

Agradecemos aos leitores que gentilmente reportaram erros: Christopher Van Der Westhuizen, pinterk, Bryan Stillwell, Elisey Shemyakin, oneveu, Igor Nikolaev, Mikhail Naganov, ghosttie, elieb, Thizz (@codyvasy), Olivier Cahagne, tronster, Cyril Mottier, Ted Marynicz, Eluan Costa Miranda, Justin Meiners, A. Piquet, Amro, Ben Terris, Chris Chokan, David Snyder, Dennis Duda, Dmitry Minsky, Graeme McCutcheon, James F McMahon, Jonathan Jayet, LFaria, Tim Garlick, Elisey Shemyakin e SysTools.

Como enviar feedback

Este livro busca a precisão. Caso encontre erros, omissões ou erros de digitação, por favor, dedique um momento para nos informar.

Ou melhor ainda, envie uma atualização para o livro por meio de um pull request, já que o código-fonte completo está disponível online no GitHub.

Reportar um problema:<https://github.com/fabiensanglard/gebbwolf3/issues>.

Enviar uma solicitação de pull request:<https://github.com/fabiensanglard/gebbwolf3/pulls>.

Envie-me um e-mail:fabiensanglard.net@gmail.com.

Obrigado :) !

Prefácio de John Carmack

Os comentários de Fabien sobre os códigos-fonte dos motores de jogos clássicos têm sido um recurso maravilhoso na internet, então fiquei muito feliz quando ele decidiu expandi-los para um livro. Embora muitas vezes ofuscado por Doom, Wolfenstein 3D ocupa um lugar significativo na história dos videogames e continua divertido de jogar hoje em dia, assim como colocar uma moeda em uma máquina de Pac-Man.

Apesar de ser de código aberto, o código de 16 bits e a linguagem assembly não são fáceis de compilar ou experimentar, por isso muito menos pessoas se interessaram por ele do que pelas bases de código posteriores. O mais notável sobre o projeto, da perspectiva atual, é o quão pequeno ele era: um pequeno diretório de arquivos de código sem dependências externas. Naquela época, eu mal confiava (com alguma razão!) nas implementações da biblioteca padrão C com as quais tínhamos que trabalhar, então quase tudo era feito nesses poucos arquivos-fonte.

Eu tinha 21 anos quando escrevi a maior parte do código e só programava em C havia um ano, então está longe de ser uma obra-prima em termos de estilo de programação, mas ainda assim há algumas coisas que foram bem feitas. Os escaladores compilados são um caso de especialização de código levada ao extremo, que, combinada com o truque de baixo nível da VGA de escrita em múltiplas colunas e as características de desempenho progressivo do ray-casting, tornam a taxa de quadros muito mais uniforme do que uma abordagem convencional. A escolha de usar ray-casting também foi uma importante decisão pragmática. Eu ainda não tinha experiência suficiente para fazer uma implementação sólida de um motor baseado em polígonos, ou mesmo em linhas. O ray-casting me levou aonde eu precisava chegar a um custo aceitável.

Então, programe a máquina do tempo para 1992.

ACHTUNG!

– John Carmack

Prefácio de Tom Hall

Tudo começou com um telefonema.

Na id Software, tínhamos tido grande sucesso com nossos jogos Commander Keen, baseados na minha infância, no meu amor por ficção científica e nos desenhos animados da Warner Brothers de Chuck Jones. John Romero, um dos quatro cofundadores da id (junto comigo, John Carmack e Adrian Carmack – sem parentesco), conversou com Paul Neurath, da Looking Glass Studios, sobre um novo jogo que eles estavam desenvolvendo (Ultima Underworld). Romero desligou o telefone e disse: "Paul disse que eles estão usando uma técnica chamada 'mapeamento de textura'". John Carmack olhou para cima e para a esquerda, pensou um pouco e disse: "Eu consigo fazer isso".

Então, começamos a trilhar o caminho rumo a esse objetivo. Hovortank 3D (também conhecido como Hovortank One) era um jogo de tiro simples, onde você dirigia através de labirintos com paredes coloridas. Dirija seu tanque com a torreta no centro inferior da tela. Atire em monstros, resgate humanos. Ele, e os jogos que se seguiram, podem ser considerados os primeiros FPS (jogos de tiro em primeira pessoa). Em seguida, veio Catacomb 3D, uma versão tridimensional de um jogo de ação 2D que criamos, mas desta vez com paredes texturizadas (em EGA de 16 cores) e uma mão lançando bolas de fogo mágicas. Às vezes, uma bola de fogo atingia uma parede e desaparecia, revelando uma sala ou caminho secreto. Estávamos então prontos para experimentar o VGA de 256 cores. Estávamos procurando um tema. Tive centenas de ideias diferentes... mas precisávamos de algo simples e brutal, assim como o motor gráfico.

Romero teve a ideia do antigo jogo para Apple II, Castle Wolfenstein. Todos nós tínhamos jogado, e parecia uma escolha natural. Depois de algumas tentativas frustradas de replicar Beyond Castle Wolfenstein (a sequência de Castle Wolfenstein), que tinha arrastamento de corpos, disfarces e coisas do tipo, optamos pelo original, mais simples. Eu criei o conjunto básico de armas, com a honra de descobrir qual seria o primeiro conjunto natural de armas de FPS, com melhorias incrementais. Trabalhávamos no jogo de 14 a 18 horas por dia, sete dias por semana. Uma noite, nos olhamos e dissemos: "Ninguém nunca fez um jogo como este antes..." Algo como: "Tudo bem?". Foi uma sensação estranha, mas empolgante. Este jogo 3D era diferente.

Commander Keen rendeu um bom dinheiro. Os jogos seguintes da série renderam muito dinheiro. Wolfenstein 3D rendeu DEZ VEZES mais. Tínhamos descoberto algo promissor.

A mente brilhante de Carmack concebeu essa inovação revolucionária de ray-casting, mudando os jogos para sempre. Hoje parece antiquada, mas na época, era o auge da tecnologia de jogos. É preciso uma mente brilhante para explicar seus segredos e métodos.

Por sorte, temos o Fabien. Com sua vasta experiência em analisar jogos clássicos e os motores de renderização por trás deles, não há ninguém melhor para mostrar a todos o homem por trás da cortina. O Grande e Poderoso Oz parece mágico, mas ele é simplesmente genial em matemática, geometria, álgebra e eficiência de algoritmos. (E, bem, não é lá essas coisas em ortografia, hehe, mas isso não importa aqui.)

Dissemos que estávamos ali para ver o mago. Fabien viu nossos sapatos vermelhos de curiosidade intelectual. Fabien se iluminou e disse: "Por que vocês não disseram isso desde o início? Isso é outra história! Entrem!" E abriu os portões.

É hora de entrar.

– Tom Hall

Designer de jogos; cofundador e ex-diretor criativo da id Software.

São Francisco, CA

Setembro de 2018

Prefácio de John Romero

Existem poucos livros como este que você tem em mãos. Trata-se de uma rara dissecação tecnológica e arqueológica de um jogo antigo, porém significativo: Wolfenstein 3D, o precursor do gênero de tiro em primeira pessoa. Há alguns outros livros que exploram a fundo a tecnologia de jogos antigos, como Retrogame Archeology, mas nenhum outro explica com detalhes técnicos exaustivos o ambiente computacional em que o jogo foi desenvolvido antes de tentar explicar seu funcionamento. Nenhum outro livro detalha o código-fonte de um jogo com uma compreensão tão completa e minuciosa que não deixa espaço para dúvidas sobre seu funcionamento.

Fabien foi exaustivo em sua análise deste importante artefato. Ele explica as dificuldades que enfrentamos naquela época para realizar raycasting e renderizar uma cena a 70 quadros por segundo. Ao longo do caminho, ele revela diversas informações valiosas sobre o desenvolvimento, que só foram obtidas por meio de questionamentos diretos aos dois únicos programadores do jogo, John Carmack e eu. O livro Masters of Doom oferece uma visão superficial da criação de Wolfenstein 3D, mas se você quer a história completa, é aqui que você a encontrará.

Não sabíamos que estávamos criando um jogo que daria início a uma indústria em 1992. Do início do projeto ao lançamento da versão shareware, nós quatro desenvolvedores levamos apenas quatro meses de trabalho. Naquela época, trabalhávamos em um ritmo frenético. Antes de Wolfenstein 3D, passamos alguns anos criando jogos e lançando-os em dois meses. Tínhamos muita prática. Quando surgiu a ideia de criar nosso próprio FPS, minha ideia de recontar a história de Castle Wolfenstein pareceu uma ótima ideia óbvia. A maioria de nós quatro adorava o clássico original de Silas Warner para Apple II. Recriar o jogo em 3D parecia que realmente poderia funcionar, então tomamos a decisão em cerca de uma hora, abandonamos nosso diretório de desenvolvimento de Commander Keen 7 (para nunca mais voltar) e criamos o diretório W3D em janeiro de 1992.

Criar Wolfenstein 3D, primeiro em Madison, Wisconsin, e depois na ensolarada Mesquite, Texas, alguns meses depois, foi muito divertido. Vivemos muitas aventuras em um curto espaço de tempo. Chegamos a viajar para encontrar Ken e Roberta Williams na Sierra e mostramos a Ken nossa homenagem ainda em desenvolvimento ao grande original, que ele também reverenciava. Wolfenstein 3D fez com que Ken Williams oferecesse US\$ 2,5 milhões para comprar nossa pequena empresa. Felizmente, o negócio não se concretizou e continuamos a desenvolver o jogo. Mudamos a empresa de...

De norte a sul. Fizemos nosso primeiro jogo VGA.

Após o lançamento, passamos mais dois meses desenvolvendo os episódios 2 a 6. Foi um período ótimo e feliz. Tanta coisa aconteceu durante o desenvolvimento do jogo, e depois do lançamento, tudo no nosso mundo parecia como quando finalmente alcançamos as estrelas, relativamente falando. DOOM nos levaria para outra galáxia em breve.

Aproveite este excelente livro. Ele contém os segredos complexos do nosso primeiro jogo de tiro em primeira pessoa. Pode não ser fácil de entender, mas precisávamos saber todas essas informações em 1992, que Fabien está ensinando a você agora, para que pudéssemos criar o FPS.

Saúde,

John

Conteúdo

Agradecimentos	5
Prefácio de John Carmack	9
Prefácio de Tom Hall	11
Prefácio de John Romero	13
Prólogo	19
1 Introdução	21
2 Hardware	27
2.1 CPU: Unidade Central de Processamento	28
2.1.1 Visão geral	28
2.1.2 O Intel 80386	29
2.1.3 Ponto Flutuante	34
2.2 RAM	39
2.2.1 Limitações do DOS	39
2.2.2 O infame Modo Real: limite de 1 MiB de RAM	40
2.2.3 O infame Modo Real: endereçamento segmentado de 16 bits	42
2.2.4 Memória Estendida	44
2.3 Vídeo	47
2.3.1 Histórico dos Adaptadores de Vídeo	48
2.3.2 Arquitetura VGA	50
2.3.3 Loucura Planar VGA	51
2.3.4 Modos VGA	54
2.3.5 Programação VGA: Mapeamento de memória	54
2.3.6 Programação VGA: Modo 12h	54
2.3.7 Programação VGA: Modo 13h	57
2.3.8 A importância do buffer duplo	60
2.4 Áudio	61
2.4.1 AdLib	63

2.4.2 Sound Blaster	64
2.4.3 Sound Blaster Pro	65
2.4.4 Fonte de som Disney	66
2.5 Ônibus	67
2.6 Entradas	69
2.7 Resumo	70
3 Equipe	71
3.1 Organização	75
3.2 Programação	78
3.3 Recursos Gráficos	83
3.4 Fluxo de Trabalho de Ativos	85
3.5 Mapas	90
3.6 Áudio	94
3.6.1 Sons	94
3.6.2 Música	95
3.7 Distribuição	97
4 Software	103
4.1 Obtendo o Código Fonte	103
4.2 Primeiro Contato	103
4.3 Visão Geral	106
4.3.1 Laço Desenrolado	106
4.4 Arquitetura	110
4.4.1 Gerenciador de Memória (MM)	112
4.4.2 Gerenciador de Páginas (PM)	115
4.4.3 Gerenciador de Vídeo (VL e VH)	118
4.4.4 Gerenciador de Cache (CA)	118
4.4.5 Gerenciador de Usuários (EUA)	119
4.4.6 Gerenciador de Som (SD)	119
4.4.7 Gerenciador de Entrada (IN)	120
4.5 Inicialização	120
4.5.1 Login	120
4.5.2 Resolvendo o Problema VGA	123
4.5.3 Carnificina Profunda	131
4.6 Fase do Menu: Renderizador 2D	132
4.7 Fase de Ação: Renderizador 3D	136
4.7.1 Vida útil de uma moldura	137
4.7.2 Vida útil de uma moldura 3D	139
4.7.3 Configuração 3D	145
4.7.4 Limpando a tela	152
4.7.5 Resolvendo o Problema da CPU	154
4.7.6 Correção do efeito olho de peixe	178

4.7.7 Desenho de Paredes	182
4.7.8 Desenhando Sprites	200
4.7.9 Saque de Armas	210
4.7.10 IA	210
4.8 Áudio e Batimentos Cardíacos	217
4.8.1 IRQs e ISRs	218
4.8.2 PIT e PIC	220
4.8.3 Batimentos cardíacos	220
4.8.4 Sistema de áudio	220
4.8.5 Música	221
4.9 Efeitos Sonoros	223
4.9.1 Efeitos sonoros: AdLib	226
4.9.2 Sistema de Fonte de Som Disney: PCM	226
4.9.3 Sistema SoundBlaster: PCM	226
4.9.4 Sistema SoundBlaster Pro: PCM estéreo	226
4.9.5 Alto-falante para PC: Ondas Quadradas	227
4.9.6 Alto-falante para PC: PCM	230
4.9.7 Alto-falante para PC: PWM	232
4.10 Entradas do Usuário	233
4.10.1 Teclado	233
4.10.2 Rato	234
4.10.3 Joystick	235
4.11 Truques	238
4.11.1 Consulta da tabela de cosseno/senoidal	238
4.11.2 FizzleFade	239
4.11.3 Paleta	247
4.12 Gerador de números pseudoaleatórios	250
4.13 Desempenho	252
5 Sequências	257
5.1 Lança do Destino	257
6 Portos	263
6.1 Super Nintendo	263
6.2 Jaguar	266
6.3 iPhone	270
Notas de desenvolvimento para iPhone 6.3.1	271
6.4 Wolfenstein 3D-VR	281
7 Epílogo	287
7.1 Onde eles estão agora?	288
Apêndices	291

U	Antes de Wolfenstein 3D	293
A.1	Hovertank Um	293 Catacumbas 3-
A.2	D	293
B	XMS vs EMS	297
B.1	EMS: Especificação de Memória Expandida	297
B.2	XMS: Especificação de Memória Estendida	299 O que
B.3	isso significou para Wolfenstein 3D	300
C	A barreira de 640 KB	301
D	CONFIG.SYS e AUTOEXEC.BAT	305
E	Coisas boas	307
F	Notas de lançamento por John Carmack	309
G	Comentário do 20º Aniversário	311

Prólogo

Nos últimos dez anos, tenho escrito artigos explicando o funcionamento interno de motores de jogos. Tudo começou em 1999, quando baixei o código-fonte recém-liberado do Quake e o abri ansiosamente. Visual Studio 6.0. Após alguns dias de luta, eu deletei. quakesrcpasta, desanimado e sem conseguir entender nada.

Alguns anos depois, deparei-me com o lendário *Livro Negro de Programação Gráfica* Por Michael Abrash. Seus artigos explicavam o panorama geral do motor gráfico do Quake e detalhavam o agora famoso sistema de Partição de Espaço Binário.¹ Conjunto Potencialmente Visível² e suas técnicas de compressão. Agora, sabendo o que esperar, voltei ao código e o compreendi profundamente.

Imaginei que muitos outros programadores pudessem ser como eu: capazes, mas desanimados pela aparente complexidade. Então, comecei a escrever "análises de código-fonte" e a publicá-las no meu site. Ao longo dos anos, escrevi mais de cinquenta artigos, selecionando jogos lendários como Doom, Quake e Out of This World. Eu abria o motor do jogo, explorava os subsistemas e a arquitetura geral, e desenhava um mapa que, com sorte, despertaria o interesse e incentivaria outros programadores aventureiros.

Compartilhar meu conhecimento foi uma experiência gratificante. O feedback dos leitores foi positivo e explicar algo em termos simples é uma excelente maneira de garantir o domínio do assunto. Isso melhorou muito minha capacidade de assimilar grandes volumes de código e minhas habilidades de comunicação. Aprendi a usar diagramas com frequência (uma imagem vale mais que mil palavras).² 10 palavras³ e, como resultado, este livro apresenta centenas delas. Essas habilidades provaram ser inestimáveis em uma carreira que, em última análise, me levou a trabalhar no Google com o Android.

Por fim, decidi levar meus artigos para o próximo nível e criei este livro, que aborda a primeira das três combinações marcantes de hardware e motor dos anos 90:

¹Capítulo 59 do livro "Graphic Programming Black Book" de Michael Abrash.

²Capítulo 64 do livro "Graphic Programming Black Book" de Michael Abrash.

³"Código: A Linguagem Oculta do Hardware e Software de Computadores", de Charles Petzold, é um excelente estudo.

por favor.

1. Wolfenstein 3D (1992) e o i386.
2. Doom (1993) e o i486-DX2.
3. Quake (1996) e o Pentium.

Pode parecer uma perda de tempo ler e escrever sobre engines "antigas" dedicadas a máquinas, compiladores e sistemas operacionais extintos, mas elas carregam um valor imenso. Não apenas estão repletas de recursos engenhosos, como também nos lembram das limitações que os programadores do passado tiveram que superar. Elas nos lembram do espírito necessário para alcançar novas fronteiras.

As coisas não mudaram muito. Hoje em dia, podemos lidar com gigabytes, aceleradores de hardware dedicados e CPUs multi-core, mas o espírito necessário para seguir em frente permanece o mesmo. Para aqueles que enfrentam dificuldades hoje, lembrem-se de que não estão sozinhos. Outros já enfrentaram dificuldades antes. Alguns alcançaram fama e outros fortuna, mas, no fim das contas, todos pertencemos a uma família de pessoas que arregaçam as mangas e tentam melhorar as coisas com trabalho árduo. Onde quer que isso os leve, orgulhem-se do seu trabalho. Orgulhem-se da sua paixão e continuem buscando a coisa certa a fazer.⁴

– Fabien Sanglard
Ligação ocasional ao passado

Sunnyvale, CA
Janeiro de 2023

⁴"Hackers: Heróis da Revolução da Computação", de Steven Levy.

Capítulo 1

Introdução

Wolfenstein 3D, lançado em 5 de maio de 1992, estabeleceu o gênero de jogos de tiro em primeira pessoa. O design do jogo, impulsionado por um motor gráfico que permitia belos gráficos de 256 cores, velocidade, alta taxa de quadros, inteligência artificial inteligente, efeitos sonoros nítidos e música envolvente, foi universalmente aclamado. Em um ano, mais de 100.000 unidades foram vendidas.¹, trazendo fama e um pouco de fortuna para a equipe que a construiu: a id Software.



No entanto, os fãs não se contentaram em apenas zerar o jogo. Movidos pelo desejo de modificá-lo e torná-lo ainda melhor, eles criaram novas formas de jogar.

¹O jogo foi distribuído via shareware.

Com seus próprios personagens e mapas, eles começaram a explorar e a fazer engenharia reversa. Em poucos meses, os formatos de recursos já eram bem conhecidos e os mods...²Foram lançadas com gráficos, efeitos sonoros, músicas e mapas alterados. No entanto, a essência do jogo — o motor 3D e os segredos de sua velocidade — permaneceram praticamente desconhecidos.

O segredo foi mantido por um motivo óbvio: um motor gráfico poderoso é um ativo essencial para uma empresa de jogos. Para superar a concorrência, é uma boa prática comercial manter outros programadores alheios ao seu funcionamento. Isso permite manter uma vantagem tecnológica, criar jogos melhores e gerar mais lucro.

No entanto, algumas pessoas dentro da id Software não viam as coisas dessa maneira. Em vez de seguirem o senso comum, queriam abraçar o entusiasmo dos jogadores e abrir completamente o código-fonte ao público. Após muito debate interno, a id Software fez o impensável: em 21 de julho de 1995, eles carregaram um arquivo zip na internet. *ftp.idsoftware.com* Contém o código-fonte completo do motor gráfico com instruções para compilá-lo.³

“

Programação não é um jogo de soma zero. SerEnsinar algo a um colega programador não tira nada de você. Porque eu Tenho prazer em compartilhar o que posso. faço isso por amor à programação.

John Carmack - Programador

”

A abertura do código fez muito mais do que apenas capacitar os programadores. Teve duas consequências imprevistas.

Primeiramente, isso permitiu que o software continuasse funcionando muito tempo depois que o hardware e o sistema operacional para os quais foi projetado fossem substituídos. Com acesso ao código-fonte, os programadores puderam manter e adaptar o motor gráfico para novos hardwares e sistemas operacionais. Vinte anos após o lançamento de Wolfenstein 3D, você ainda pode jogar em qualquer dispositivo com um processador, alguma memória RAM e um framebuffer.

Em segundo lugar, criou uma janela de volta no tempo para 1991. Tendo analisado motores gráficos complexos como Quake III e Doom III em *fabiensanglard.net* Eu pensei que apenas daria uma olhada superficial no motor gráfico de Wolfenstein 3D e em sua tecnologia de raycasting "simples". Quando, por curiosidade, decidi me aprofundar no assunto, algo me chamou a atenção e não consegui parar de ler. Quanto mais eu lia, mais percebia que a máquina em questão, o IBM PC, foi projetada para trabalho de escritório, e não para jogos. Ela foi feita para processar números inteiros e exibir imagens estáticas.

²Versão MODIFICADA.

³Eles não eram totalmente loucos, haviam construído um novo motor de jogo que tornou Wolfenstein 3D obsoleto: Doom foi lançado em 10 de dezembro de 1993.

Processadores de texto e aplicativos de planilha. O que é um software?4O que a Microsoft fez em 1991 não foi apenas programar uma máquina – eles reaproveitaram uma ferramenta criada para trabalhos de escritório e a transformaram na melhor plataforma de jogos do mundo.

Mas por que se dar a tanto trabalho? Afinal, se você fosse uma empresa de jogos e quisesse fazer videogames, teria consoles dedicados a essa tarefa específica. O Mega Drive, o Super Nintendo e o Neo Geo tinham motores de sprites que, apesar de limitações como tamanho e número, permitiam o movimento de algo na tela simplesmente atualizando seus sprites. x, y coordenadas. Eles conseguiam gerar animações suaves a 60 quadros por segundo com facilidade, tinham controles, um sistema de áudio para som e música e eram homogêneos (por exemplo, todos os SNES eram iguais). Se você ainda quisesse usar um computador pessoal para um jogo, por que não usar um Amiga 500, que vinha equipado com coprocessadores projetados para animação?

A razão se resume em uma palavra: framebuffer. O tipo de jogo que a id Software queria criar não poderia ser feito com um motor de sprites ou truques de um processador gráfico.5Eles queriam revolucionar o mundo dos jogos, proporcionando uma experiência imersiva em três dimensões. Para isso, precisavam desenhar a tela inteira, pixel por pixel, em um framebuffer antes de enviá-la para o monitor.

Para desenhar todos esses pixels, eles precisavam de uma CPU poderosa, e um PC superava qualquer console no mercado. Sem Amiga.6Mesmo com seu coprocessador, ele poderia rivalizar com um PC em termos de potência bruta.

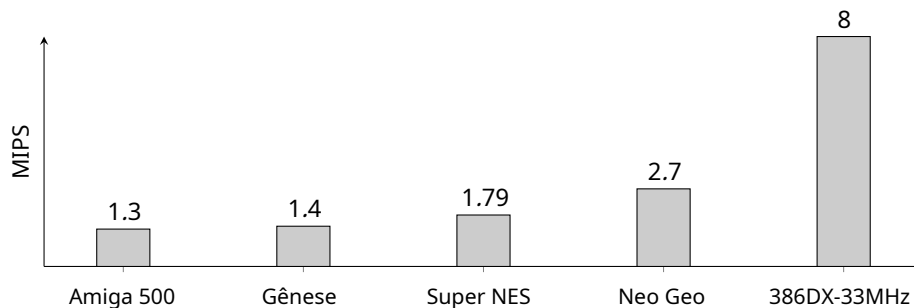


Figura 1.1:Consoles7Comparação de CPUs com PCs e MIPS8.

4Outras empresas, como a Origin Systems e a LucasArts, também estavam fazendo coisas incríveis.

5Apelido de um poderoso coprocessador Amiga que permitia operações no nível hsync.

6Jimmy Maher apresenta uma teoria interessante em seu livro "The Future was here: The Commodore Amiga": as pessoas queriam jogar jogos de tiro em primeira pessoa, algo que a arquitetura do Amiga não permitia. Essa incapacidade acabou levando ao fracasso do produto mais vendido da Commodore.

Com seu processador rápido e framebuffer de 256 KiB, um PC de 1991 parecia promissor a princípio. No entanto, havia três problemas aparentemente intransponíveis, obstáculos a superar:

- O sistema de vídeo (chamado VGA) não suportava buffer duplo. Não era possível ter animações suaves sem artefatos desagradáveis chamados "rasgos" na tela.
- A CPU só conseguia realizar operações com números inteiros, mas os cálculos em 3D exigiam o controle de frações.
- O alto-falante do PC, o dispositivo de som padrão, só conseguia produzir ondas quadradas, resultando em uma série de "bipes" que eram mais irritantes do que qualquer outra coisa.

Além desses grandes obstáculos, havia ainda mais desafios:

- O modo de endereçamento da RAM não era plano, mas segmentado, resultando em aritmética de ponteiros complexa e propensa a erros.
- Os pixels VGA não eram quadrados: o framebuffer era esticado verticalmente quando transferido para a tela.
- O ecossistema de áudio era fragmentado. Cada um dos vários sistemas de som tinha capacidades e expectativas diferentes.
- A máquina só conseguia endereçar 1 MB de RAM. Para ir além disso, era necessário entrar em um ecossistema fragmentado de drivers.
- O barramento era lento e a E/S com a VRAM era um gargalo. Era praticamente impossível escrever um framebuffer completo a 70 quadros por segundo.

No geral, parecia que a máquina estava ruína. Os computadores eram obrigados a fazer coisas tediosas. Mas muitos ao redor do mundo não aceitaram isso e mexeram no hardware para alcançar resultados inesperados. Como eles fizeram isso é o que acontece. *razão de ser* deste livro. Optei por dividir este livro em três capítulos:

- Capítulo 2: O Hardware. Os cinco componentes de um PC de 1991.
- Capítulo 3: A Equipe¹⁰As pessoas que estão ultrapassando os limites.
- Capítulo 4: O Software. O motor de jogo Wolfenstein 3D.

Ao mostrar primeiro as limitações de hardware, espero que os programadores desenvolvam uma apreciação pelo software e por como ele contornou os obstáculos, transformando, por vezes, as limitações em vantagens.

⁷O Amiga 500, o Mega Drive e o Neo Geo possuem um processador Motorola 68000 rodando a 7,16 MHz, 7,6 MHz e 12 MHz, respectivamente. O Super NES utiliza um processador WDC 65816, que é uma versão de 8/16 bits do 6502 rodando a 3,58 MHz.

⁸Um milhão de instruções por segundo.

⁹O título deste livro poderia ter sido "A Máquina Impossível".

¹⁰Este é um livro de engenharia. Para uma perspectiva mais humana, leia a obra-prima de David Kushner: "Masters of Doom".

Curiosidades: O nome "Wolfenstein 3D" foi inspirado no jogo "Castle Wolfenstein" para Apple II, lançado em 1981 por Silas Warner.

//

Batizamos o jogo de Wolfenstein 3-D porque o Castle Wolfenstein original teve uma sequência chamada Beyond Castle Wolfenstein, que foi o segundo jogo da série. Nossa versão de Castle Wolfenstein seria a terceira e seria em 3D. Usamos o mesmo sistema de nomenclatura para Catacomb, Catacomb 2 e Catacomb 3-D.

João Romero

//



Figura 1.2: Tela de introdução de "Castle Wolfenstein"

A versão da Apple era discreta (o estilo "Carnificina Profunda" de Wolfenstein era um claro afastamento do tema original), mas realmente se destacou graças ao seu uso inédito de vozes digitalizadas.

Inicialmente, a equipe acreditava que não poderia usar o nome Wolfenstein devido a questões comerciais.

questões de marca registrada. Eles sugeriram vários títulos possíveis, mas finalmente conseguiram localizar a senhora em Baltimore, Maryland, que detinha a marca registrada Wolfenstein e a compraram dela por US\$ 5.000 em 1992.

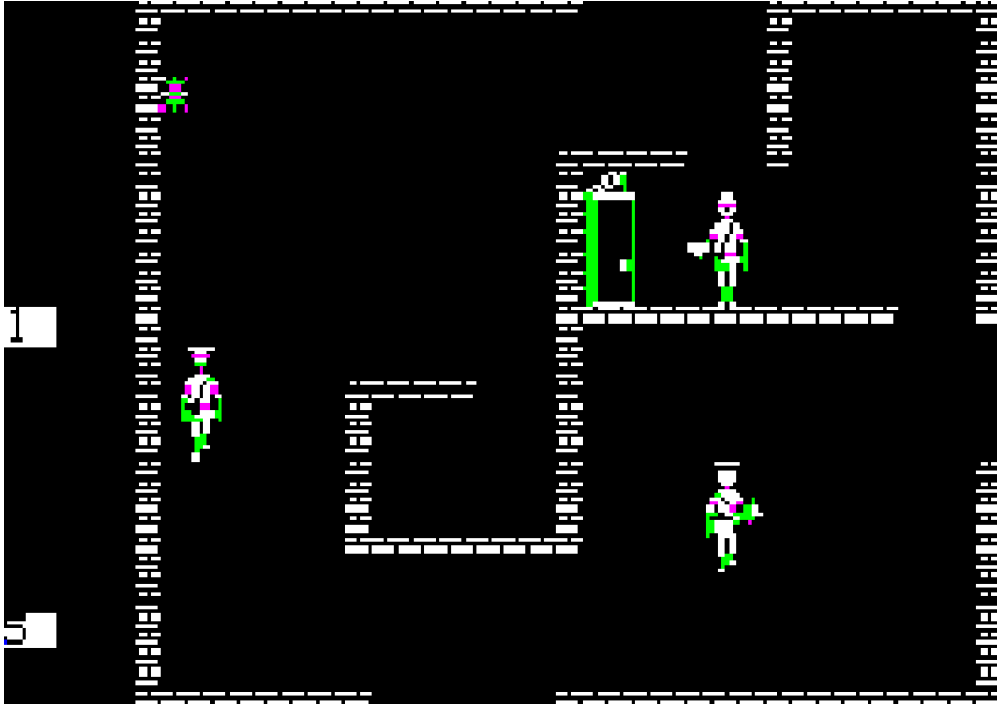


Figura 1.3: Jogabilidade furtiva em "Castle Wolfenstein".

Capítulo 2

Hardware

Para estudar o IBM PC, a maneira mais fácil é dividi-lo em partes menores. Cinco subsistemas formam um pipeline: Entradas, CPU, RAM, Vídeo e Áudio.

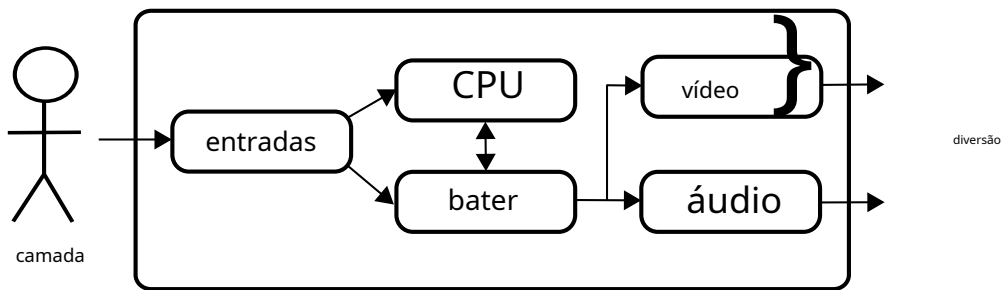


Figura 2.1: Pipeline de hardware.

Havia muita resistência, já que os fabricantes ainda não haviam se adaptado à indústria de jogos. A qualidade das peças variava de ruim a péssima, chegando a ser praticamente impossível de lidar.

Estágio	Qualidade
BATER	Suportável
Vídeo	Impossível
Áudio	Muito ruim
Entradas	OK
CPU	Impossível

Figura 2.2: Qualidade dos componentes para um motor de jogo.

2.1 CPU: Unidade Central de Processamento

Em 1991, havia 56 milhões de PCs nos EUA.¹O desempenho dessas máquinas era tão fortemente determinado pela CPU que um PC era referido não pela sua marca ou GPU.²mas sim pelo chip principal interno. Se um PC tivesse um Intel 80386 ou equivalente, era chamado de "386". Se tivesse um Intel 80286, era chamado de "286".

2.1.1 Visão geral

O principal fabricante de CPUs era a Intel, com sua linha de microprocessadores x86.³Os computadores baseados no processador 80286 de 16 bits não venderam bem, mas seus sucessores, o 80386, lançado em 1985, foram um enorme sucesso.⁴Como resultado, Wolfenstein 3D foi projetado para rodar em um PC com um processador 386, com desempenho reduzido (mas ainda jogável) em um 286.

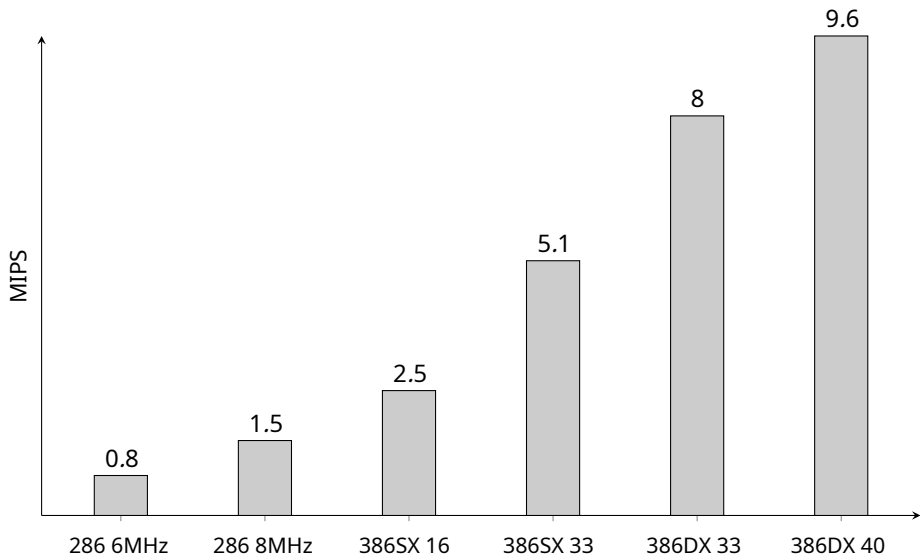


Figura 2.3: Comparações de CPUs com MIPS

¹"Computadores". Enciclopédia Collier. Vol. 7, 1992: 114, 129.

²Ainda não existia GPU. O termo foi cunhado pela Nvidia em 1999, que comercializou a GeForce 256 como "a primeira GPU do mundo", ou Unidade de Processamento Gráfico.

³Segundo o Relatório Anual da Intel Corporation, 73% dos PCs vendidos em 1989 possuíam um processador Intel.

⁴Estima-se que 3,74 milhões de sistemas baseados no processador 386 foram vendidos em 1990, de acordo com a Dataquest na revista PC Magazine de fevereiro de 1992.

⁵Coleção de benchmarks de PC de Roy Longbottom: <http://www.roylongbottom.org.uk/mips.htm#anchorIntel2>.

Curiosidades: Um Core i7 Extreme Edition 980X de 3.330 MHz de 2010 opera a 147.600 MIPS.

A Intel fabricou duas versões do 386: o 386-SX e o 386-DX. Eram processadores idênticos, porém a versão DX era quase duas vezes mais potente que a SX (no gráfico...). 386 SX 33MHz e o 386 DX 33MHz estão respectivamente em 5,1 e 8 MIPS). Isso se deve a um dado

ênibus A distância entre a CPU e a RAM era duas vezes maior no DX (32 bits contra 16 bits). A unidade de barramento do 386-SX permitiu que os fabricantes reutilizassem chips de placa-mãe projetados para o 286, que possuía uma unidade de barramento de 16 bits, reduzindo ainda mais os custos. Apesar de sua inferioridade, o SX fez sucesso porque era mais barato e muitas pessoas não sabiam o que era um barramento; elas queriam "um 386".

Curiosidades: Para ajudar a atender à demanda, a Intel contratou a AMD para fabricar os processadores 8088, 8086 e 80286 CPUs. Processos judiciais, que se estenderam por várias décadas, ocorreram em relação ao i386. A milhaidade variava com os "clones". A AMD produziu o 386 mais rápido (Am386DX 40MHz). A reputação da repCyrix era instável entre os jogadores devido a um 486SLC quase fraudulento (um 386SX com 486 instruções e um pequeno cache L1) e um 6x86 que tinha um desempenho péssimo em Quake.

2.1.2 O Intel 80386

O A jornada do projeto ao silício não foi um passeio tranquilo para a 80386. Começou como um projeto paralelo para uma pequena equipe em San Diego, enquanto alguns funcionários em Portland trabalhavam no projeto principal. O Intel i960, uma CPU que utiliza um novo conjunto de instruções capaz de executar linguagens de alto nível e hardware de coleta de lixo em memória. Quando o time de Portland enfrentou dificuldades devido ao desempenho, o 386 passou de patinho feio a rei.⁶



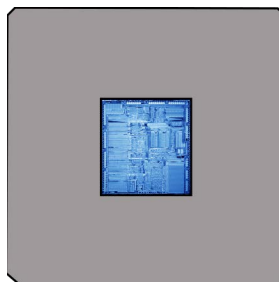
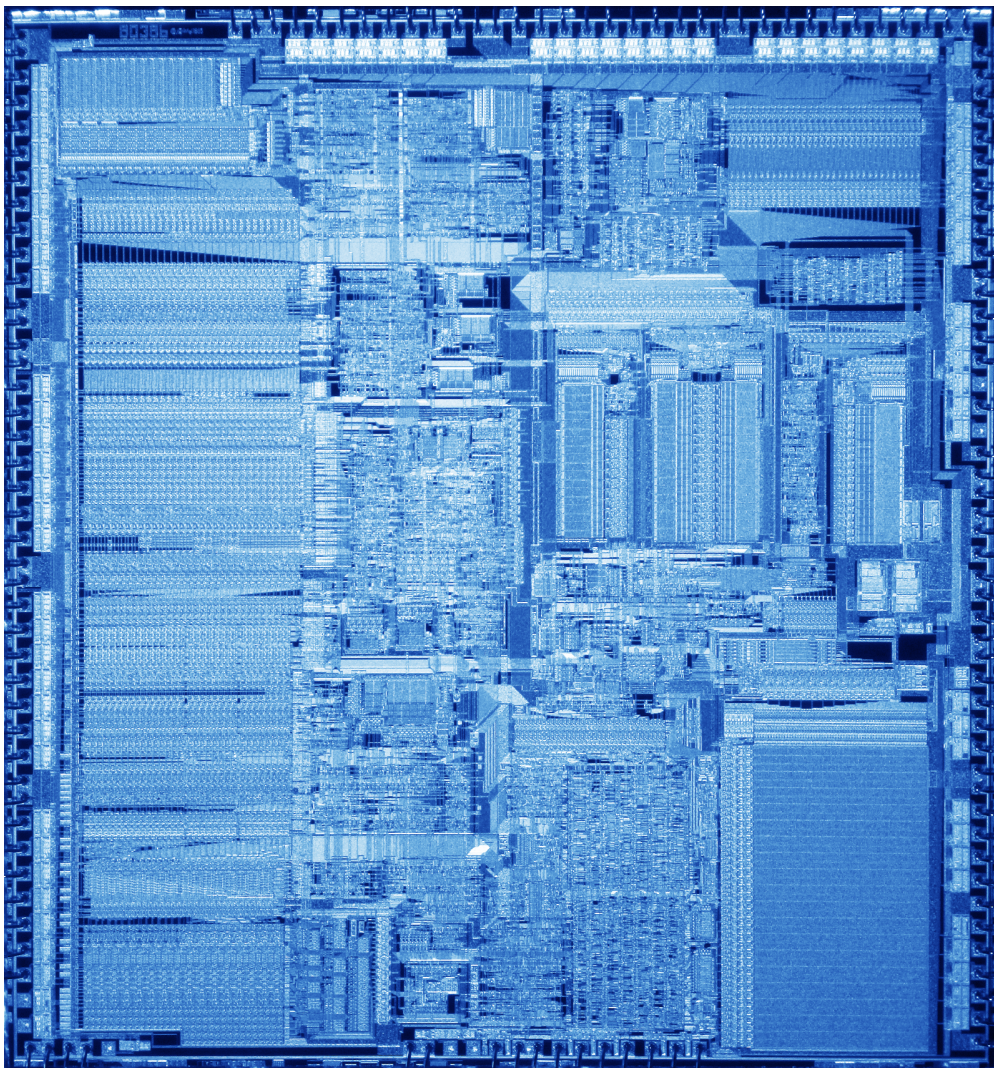
Dois escolhas feitas no projeto do 386 contribuíram para o seu sucesso. Primeiro, os projetistas decidiram acatar o feedback dos programadores e abandonaram a ideia de usar um novo conjunto de instruções. Como resultado, o 386 é totalmente compatível com o 286. Segundo, eles conseguiram adicionar um modo de operação de 32 bits que resolveu muitos dos problemas de endereçamento de memória do 286.

Como o 80286 e o 80386 usavam um 1,5 processo m. A superfície aumentada (de 49 mm² para 104 mm²) permitiu à Intel incluir 275.000 transistores em vez de 134.000.

Curiosidades: O 286 era bastante impopular entre programadores e projetistas de hardware. Bill Gates o chamou de "sem cérebro".⁷ para sistemas operacionais e Steve Morris (co-arquiteto do Intel 8086) chamou-o de "veneno de software".

⁶Painel de História Oral sobre o Projeto e Desenvolvimento do Microprocessador Intel 386.

⁷Dewar, Robert BK; Smosna, Matthew (1990). Microprocessadores: Uma visão do programador.



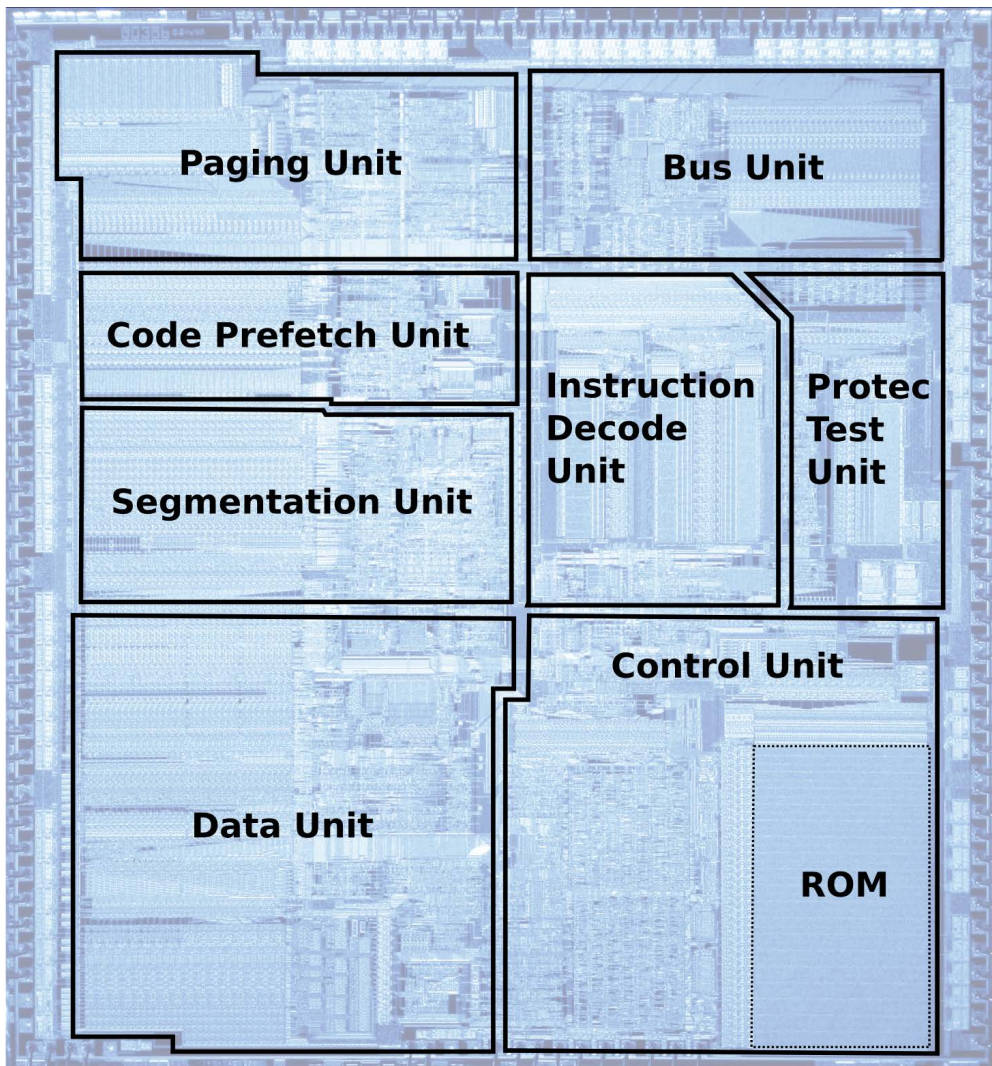
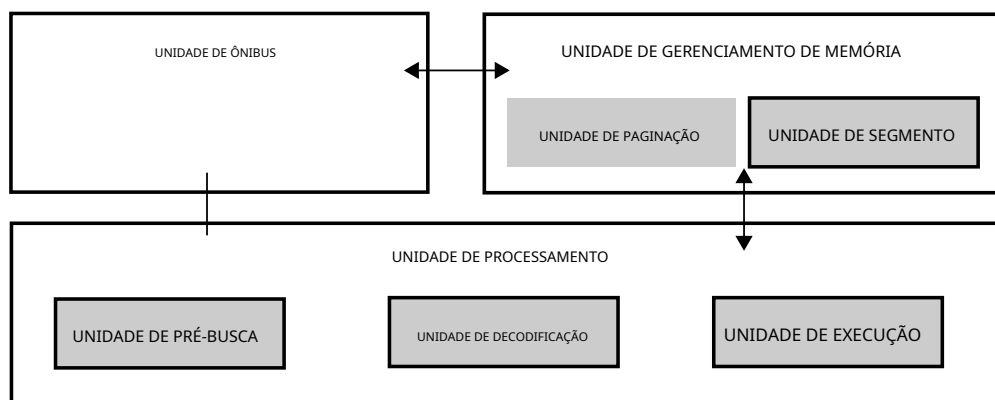


Figura 2.4. Layout do chip Intel 80386-DX de 16 MHz⁸

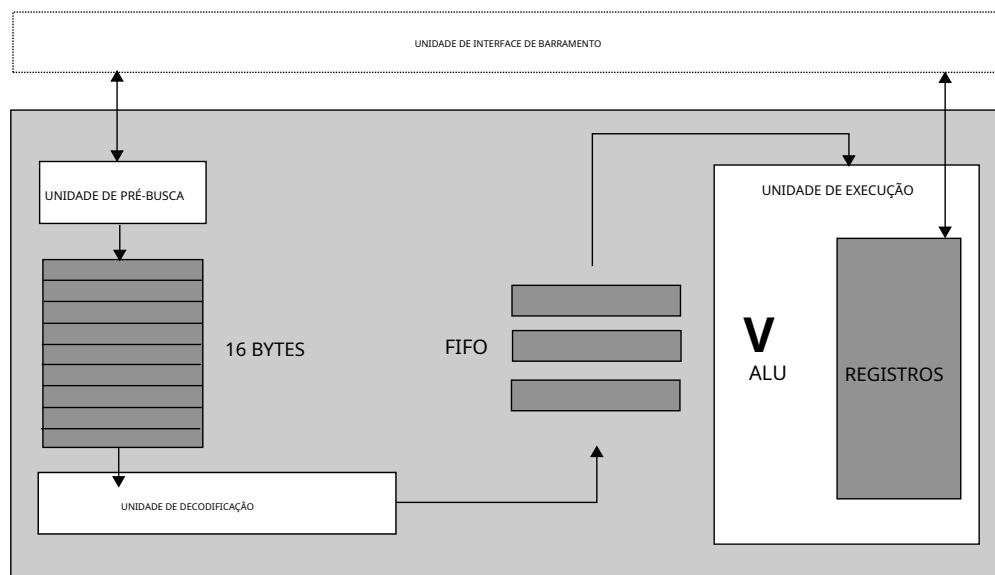
Se você estiver segurando uma cópia física deste livro no formato 9,25" x 7,5", a embalagem da CPU tem dimensões de 36,8 x 36,8 mm e o chip mede 10 x 10 mm, em escala 1:1.

Apesar da aparente complexidade, o 80386 pode ser resumido em três sistemas e um pipeline de instruções de três estágios.

⁸Fonte: "The Intel 80386 - Architecture and Implementation" de Khaled A. El-Ayat e Rakesh K. Agarwal.



A única diferença entre um SX e um DX está na unidade de barramento. O SX possui um barramento de 16 bits, o que permitiu aos fabricantes de PCs reutilizar o projeto das placas-mãe do 286, reduzindo significativamente o preço. O DX, por sua vez, possuía uma unidade de barramento de 32 bits.



As três unidades do grupo de execução formam um pipeline de três estágios: Pré-buscar, decodificar e Executar. A unidade de pré-busca é ativada quando a unidade de execução está em execução, mas não está sendo usada. O barramento e busca instruções em uma fila de 16 bytes. O pré-buscador é linear e não pode prever o resultado de um desvio. Como resultado, um salto (JMP) a instrução aciona uma descarga do todo o processo. As instruções percorrem o pipeline e são decodificadas pela Unidade de Decodificação: a result O resultado da operação de decodificação é armazenado em um FIFO de três elementos, de onde é recuperado por

a Unidade de Execução.

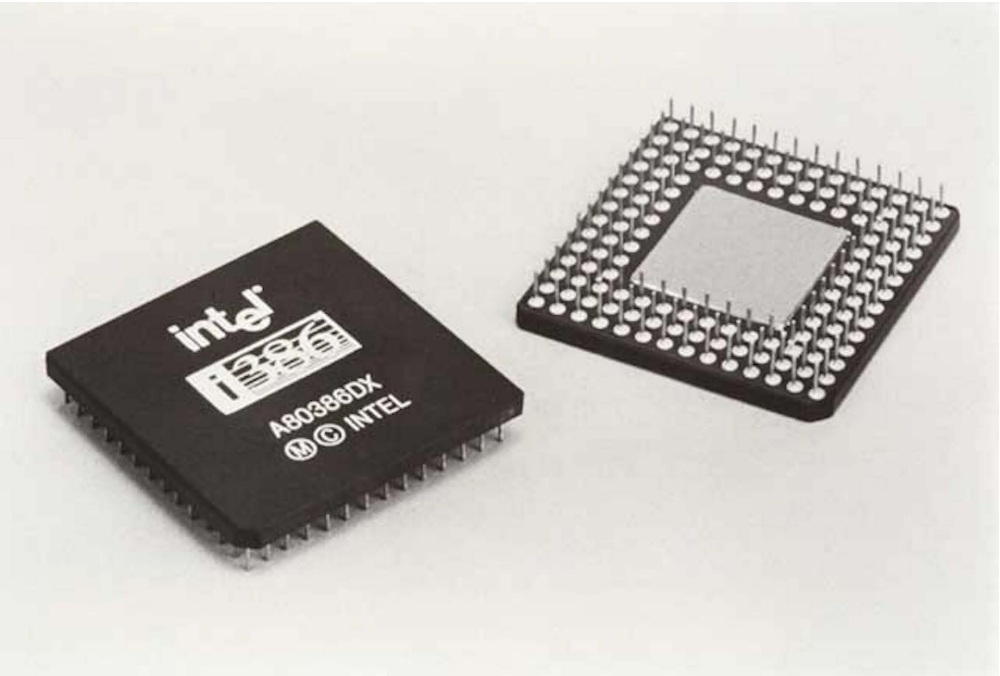
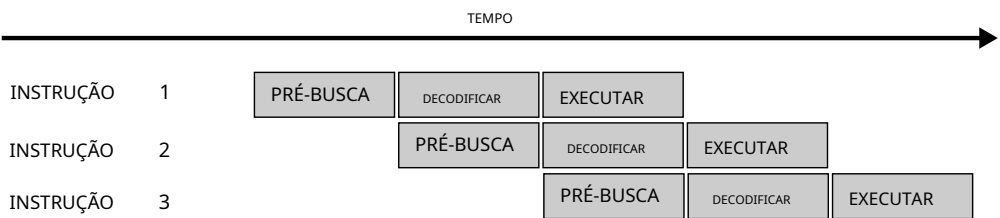


Figura 2.5: O Intel 386, com dimensões de 10 mm por 10 mm, contém 275.000 transistores.

Do ponto de vista da programação, uma CPU 386 pode ser resumida pelos seguintes elementos:

- Unidade Lógica Aritmética em execução adicionar, subtrair, multiplicar e cetera.
- 16 registros:
 - 32 bits Uso geral RRegistros: EAX, EBX, ECX, EDI

- Registros de índice de 32 bits: ESI, EDI, EBP, ESP
 - Registros de segmento de 16 bits: CS, DS, ES, FS, GS, SS
 - Registro de status de 16 bits
 - Contador de programa de 32 bits: EIP
- Um barramento de endereços de 32 bits para até 4 GB de RAM endereçável plana
 - Unidade de paginação de memória

Curiosidades: Apesar de seu design para processamento em série, o 386 não consegue realizar uma operação em menos de dois ciclos. Mesmo uma operação simples leva mais de dois ciclos. ADICIONAR reg, regouRegistro INCSão necessários dois ciclos de clock. Isso se deve à ausência de um cache SRAM no chip e a uma unidade de decodificação lenta.

Tipo de instrução	Relógios
ADICIONAR reg16, reg16	2
INC reg16	2
IMUL reg16, reg16	12-25 ⁹
IDIV reg16, reg16	27
MOV [reg16], reg16	4
SAÍDA [reg16], reg16	25
EM [reg16], reg16	26

Figura 2.6:386 custos de instrução¹⁰

2.1.3 Ponto Flutuante

Toda essa potência de CPU não era necessariamente útil para programar um jogo. Para realizar cálculos trigonométricos para efeitos 3D, o motor precisa controlar a parte fracionária de cada operação. Isso pode não parecer um problema, já que a linguagem de programação C possui um tipo (flutuador)precisamente para esse propósito. Mas, na prática, isso era um problema, e para entendê-lo precisamos entender comoflutuadorfunciona.

Como David Goldberg escreveu de forma memorável, *"A aritmética de ponto flutuante é considerada um assunto esotérico por muitas pessoas."*¹¹Não poderia concordar mais. No entanto, é importante entender para compreender plenamente o quão útil isso é para programar um motor 3D. Na linguagem C, flutuadorOs s são contêineres de 32 bits que seguem o padrão IEEE 754. Seu propósito é armazenar e permitir operações de aproximação de números reais. Os 32 bits são divididos em três seções:

⁹Nem todas as multiplicações são iguais. O 80386 usa um algoritmo de multiplicação com saída antecipada (early-out). O número real de ciclos de clock depende da posição do bit mais significativo no multiplicador otimizador.

¹⁰Manual de referência do programador Intel 80386 - 1986.

¹¹"O que todo cientista da computação deve saber sobre ponto flutuante", por David Goldberg.

- 1 bit S para o sinal
- 8 bits E para o expoente
- 23 bits M para a mantissa



Figura 2.7: Componentes internos de ponto flutuante



Figura 2.8: As três seções de um número de ponto flutuante.

A forma como os números são armazenados e interpretados geralmente é explicada pela fórmula:

$$(-1)^S \cdot 2^{E-127}$$

Figura 2.9: Como todo mundo odeia que expliquem o conceito de ponto flutuante.

Embora correta, essa forma de explicar a notação de ponto flutuante geralmente deixa os programadores completamente perdidos. Culpo essa notação terrível por desencorajar legiões de programadores, assustando-os a ponto de nunca mais se darem ao trabalho de entender como o ponto flutuante realmente funciona. Felizmente, existe uma maneira melhor de explicá-la. Em vez de um expoente, pense em uma janela entre dois inteiros consecutivos que são potências de dois. Em vez de uma mantissa, pense em um deslocamento dentro dessa janela.



Figura 2.10: Componentes internos alternativos de ponto flutuante.

A janela indica em quais duas potências de dois consecutivas o número estará. De $[2^{-127}, 2^{-126}]$, ..., $[0.5, 1]$, $[1, 2]$, $[2, 4]$, e até $[2^{127}, 2^{128}]$. O deslocamento divide a janela em $2^{23} = 8,388,608$ baldes. Com a janela e o deslocamento, você pode aproximar um número. A janela é um excelente mecanismo para proteger contra transbordamentos. Uma vez

Se você atingiu o máximo em uma janela (por exemplo, [2,4]), você pode "flutuar" para a direita e representar o número na próxima janela (por exemplo, [4,8]). Isso custa apenas um pouco de precisão.

Curiosidades: Quanta precisão é perdida quando a janela cobre um intervalo maior? Vejamos um exemplo com a janela [1,2], onde os 8.388.608 deslocamentos cobrem um intervalo de 1, o que resulta em uma precisão de $(2-1)/8388608 = 0.00000011920929$. Na janela [2048,4096], os 8388608 deslocamentos cobrem uma gama de $(4096-2048) = 2048$ o que proporciona precisão $(4096-2048)/8388608 = 0.0002$.

A próxima figura ilustra como o número 6,1 seria codificado. A janela deve começar em 4 e se estender até a próxima potência de dois, 8. O deslocamento fica aproximadamente na metade da janela.

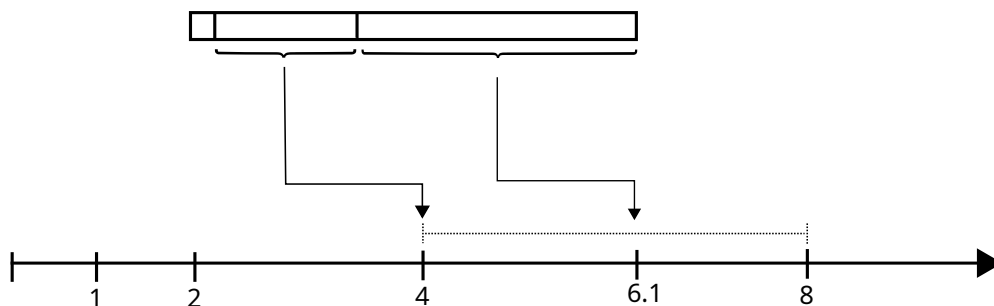


Figura 2.11: Valor 6.1 aproximado com ponto flutuante

Aqui está um exemplo detalhado que calcula a representação em ponto flutuante de um número que conhecemos bem: 3,14.

- O número 3,14 é positivo $\rightarrow S=0$.
- O número 3,14 está entre as potências de dois 2 e 4, portanto a janela flutuante deve começar em $2^1 \rightarrow E=128$ (veja a fórmula onde a janela está $2^{(E-127)}$).
- Finalmente, existem 2^{23} deslocamentos disponíveis para expressar onde 3,14 se encontra no intervalo [2-4]. Está em $3,14/2 = 0.57$ dentro do intervalo que cria o deslocamento $M=2^{23} * 0.57 = 4781507$

Que em binário se traduz para:

- $S = 0 = 0b$
- $E = 128 = 10000000b$
- $M = 4781507 = 10010001111010111000011b$

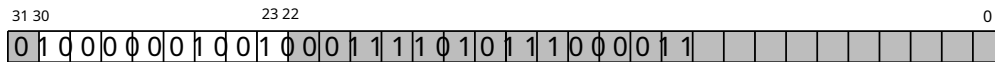


Figura 2.12:3.14 Representação binária de ponto flutuante.

O valor 3,14 é, portanto, aproximado para 3,1400001049041748046875.

O valor correspondente com a fórmula complexa:

$$(-1)_0 * 1.57 * 2_{(128-127)} = 3.14$$

E, finalmente, a representação gráfica com janela e deslocamento:

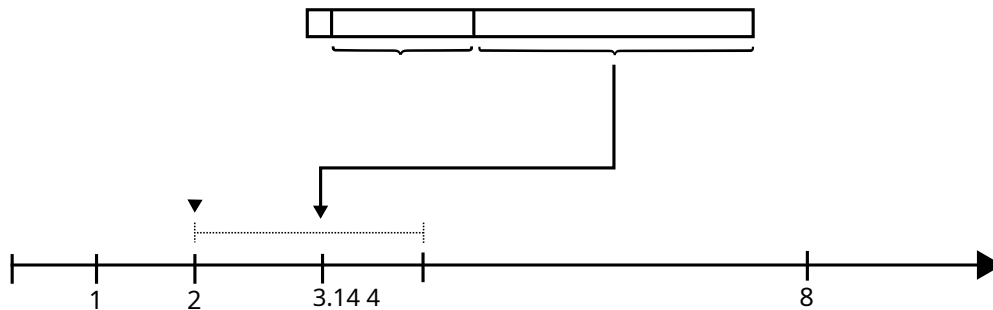


Figura 2.13:3.14 janela e deslocamento.

A aritmética de ponto flutuante é uma ferramenta poderosa. Ela pode representar valores muito pequenos ou enormes, mantendo o controle das partes fracionárias de um número, e também proteger contra estouro de capacidade, ajustando a janela de forma flutuante quando necessário.

A computação de ponto flutuante é prática, mas a desvantagem é que também é computacionalmente dispendiosa. A razão é simples. Para somar, subtrair, multiplicar ou dividir dois números, ambos precisam ser expressos na mesma janela. Isso significa converter um número para a representação usada pelo outro, geralmente com uma precisão maior que 32 bits (tipicamente 80 bits em FPU's da Intel).¹²

Isso não é um problema. **m quando tudo** está embutido em um fluxo de hardware unidade de ponto de conexão
Mas isso é um grande problema para o 386. Se você consultar o diagrama de arquitetura, perceberá que ele possui apenas uma ULA (Unidade Lógica Aritmética). *Um 386 não possui uma unidade de ponto flutuante (FPO) de hardware..* Se flutuador As operações são encontradas no código e emuladas no software pelo compilador, resultando em

¹²Para compreender plenamente o quanto processa Para entender o que uma FPU faz, é útil ler a implementação do software. Achei o Berkeley SoftFloat útil.

com um processamento extremamente lento – tão lento que não é utilizável para nada em tempo real.

Curiosidades: Já que as operações de ponto flutuante eram tão lentas, por que a linguagem C acabou com... flutuadoredobrotipos? Afinal, a máquina usada para inventar a linguagem (PDP-11) não tinha uma unidade de ponto flutuante! O fabricante (DEC) havia prometido a Dennis Ritchie e Ken Thompson que o próximo modelo teria uma.¹³ Sendo entusiastas da astronomia, eles decidiram adicionar esses dois tipos ao seu idioma.

Quem realmente quisesse uma unidade de ponto flutuante por hardware podia comprar uma. Nos anos 90, os únicos interessados seriam cientistas (segundo a visão de mercado da Intel). O chip i387 era comercializado como um "Coprocessador Matemático". O desempenho era mediano e o preço, exorbitante.¹⁴ mas venderam extremamente bem.

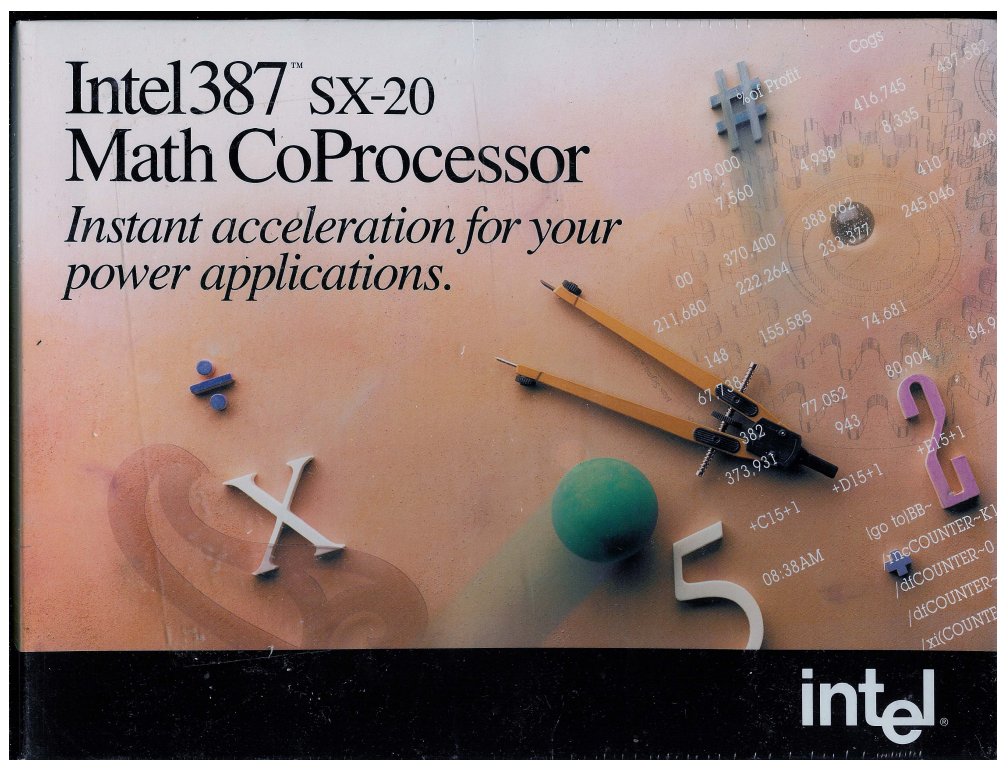


Figura 2.14: Anúncio da Intel de 1991 para um "Coprocessador Matemático" i387.

¹³O Desenvolvimento da Linguagem C por Dennis M. Ritchie.

¹⁴US\$ 200 em 1991 equivalem a US\$ 374 em 2018.

Uma possível solução para a falta de uma unidade de ponto flutuante por hardware seria multiplicar um número inteiro por 100 ou 1000 para realizar operações fracionárias e, em seguida, dividir para retornar aos números inteiros. Infelizmente, isso não é possível. Em um 386, uma multiplicação (imul) leva de 12 a 25 ciclos e uma divisão (div) é ainda pior com 27 ciclos.¹⁵

Como resultado, um designer de motor de jogo se viu numa situação delicada, com duas soluções parciais para o seu problema: números inteiros, que eram rápidos, mas não precisos o suficiente, e números de ponto flutuante, que eram precisos, mas não rápidos o suficiente.

2,2 RAM

Os primeiros processadores da família Intel x86 foram projetados em 1976. Numa época em que a memória RAM era muito cara, os processadores 8080 e 8086 possuíam registradores de 16 bits com um barramento de endereços de 20 bits capaz de endereçar 1 MiB.¹⁶ de RAM. É difícil enfatizar o quão grande era 1 MiB de RAM na década de 70, mas, como exemplo, o Apple II e o Commodore 64 eram vendidos com 64 KiB.¹⁷ o que era suficiente para escrever e executar coisas incríveis. Registradores de dezesseis bits e um barramento de endereços de 20 bits eram mais do que suficientes, embora a programação fosse difícil e exigisse a combinação de dois registradores para construir um ponteiro.

Em 1986, o hardware havia ficado mais barato e a Intel abandonou a arquitetura antiga com seus processadores 286 e 386. Essas novas CPUs podiam ser colocadas no chamado "modo protegido", apresentando um barramento de endereços de 24 bits para até 16 MiB de RAM plana, protegida por uma MMU (Unidade de Gerenciamento de Memória).¹⁸ O 386 também possuía registradores de 32 bits em modo protegido. Para garantir que programas antigos ainda pudessem ser executados, ambos os processadores podiam ser colocados em "modo real", que replicava o funcionamento dos Intel 8080 e 8086: registradores de 16 bits, barramento de endereços de 20 bits, resultando em 1 MiB de RAM endereçável com endereçamento segmentado.

Por motivos de compatibilidade, todos os PCs precisam iniciar em modo real. Você pode supor que os programadores da década de 90 prontamente mudaram a CPU para o modo protegido para liberar todo o potencial das máquinas e abandonar o modo real, que já durava 20 anos. Infelizmente, havia um grande obstáculo: o sistema operacional MS-DOS da Microsoft Corporation.

2.2.1 Limitações do DOS

A Microsoft Corporation valorizava muito os aplicativos executados em seus sistemas operacionais. Como prioridade de negócios, eles eram inflexíveis quanto a nunca quebrar nada com um novo sistema.¹⁹

¹⁵Instruções da ZSmith.com em http://zsmith.co/intel_i.html#imul.

¹⁶Este livro usa a notação IEC, onde MiB é 2²⁰e MB é 10⁶.

¹⁷Este livro usa a notação IEC, onde KiB é 2¹⁰e kB é 10³.

¹⁸Unidade de gerenciamento de memória

¹⁹"Histórias de Compatibilidade de Aplicativos", Coisa Velha e Nova por Raymond Chen.